OpenClaw Chassis Control

OpenClaw Chassis Control

1. Course Content
   - Course Overview
   - Learning Objectives
2. Preparation
3. Case Demonstration
   - 3.1 Start OpenClaw Interaction
   - 3.2 Chassis Movement Control
   - 3.3 Chassis Omnidirectional Movement Control
   - 3.4 Chassis In-Place Steering
4. Source Code Analysis
   - 4.1 `set_cmdvel` — Set Chassis Motion Velocity (Underlying Implementation of MoveWithSpeed)
   - 4.2 `move_by_odom`

1. Course Content

Course Overview

- OpenClaw chassis control is the basic feature of robot motion control. It provides 2 chassis control tools through the MCP interface: MoveWithSpeed (velocity control) and Move (odometry closed-loop control movement)
- Supports omnidirectional chassis control, capable of controlling the X/Y direction linear velocities and Z-axis angular velocity simultaneously, achieving forward/backward movement, left/right translation, and in-place rotation
- Achieves precise distance control and angle control through odometry (Odom) feedback, ensuring motion accuracy

Learning Objectives

1. Understand the functions and uses of the 3 chassis control interfaces provided by OpenClaw
2. Master the interaction methods for chassis movement, omnidirectional movement, and steering through WebChat

2. Preparation

- Start the MCP service

```
ros2 launch yahboomcar_bringup mcp_service.launch.py
```

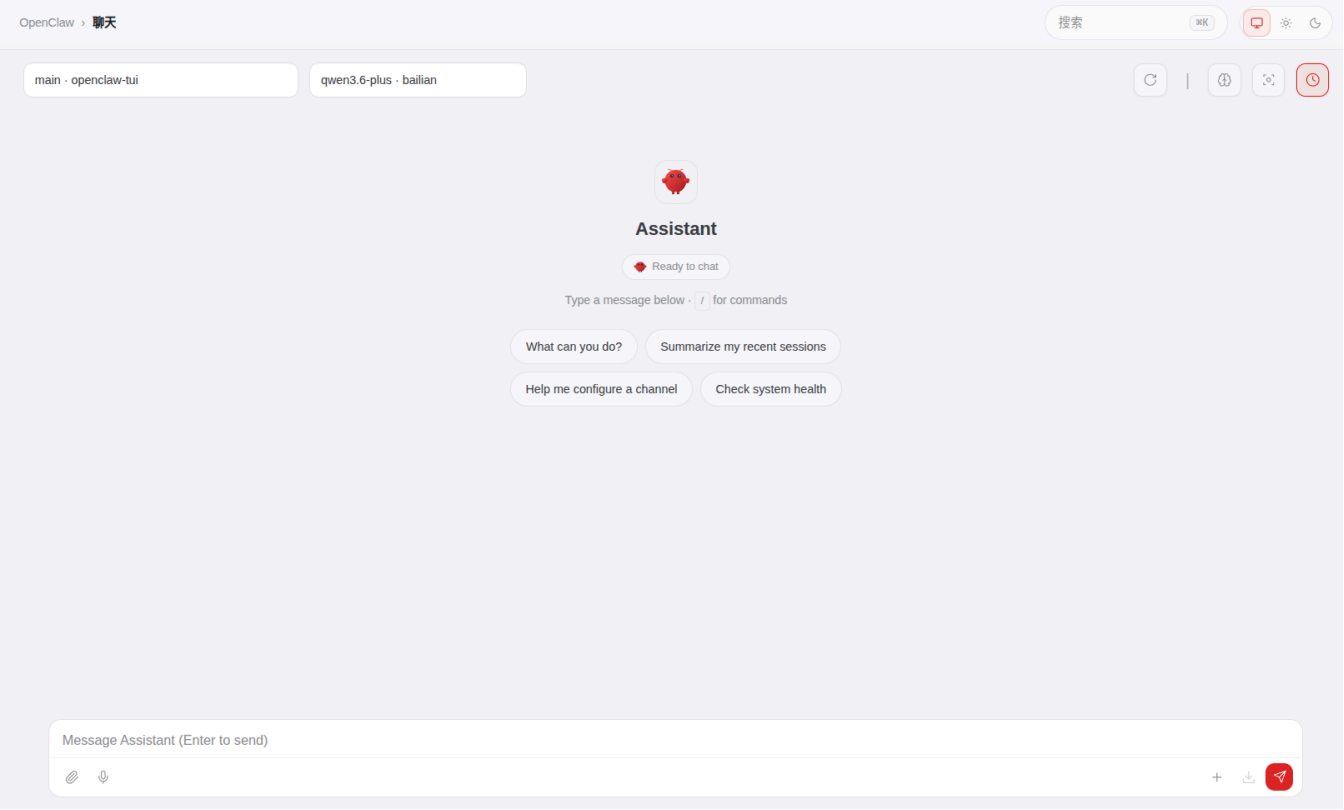
- If you need voice replies, you need to add the use_tts parameter: `use_tts:=true`, otherwise it can be omitted

No.	Tool Name	Function Description
1	MoveWithSpeed	Control the robot's omnidirectional chassis velocity
2	Move	Control the robot chassis to move a specified distance

3. Case Demonstration

3.1 Start OpenClaw Interaction

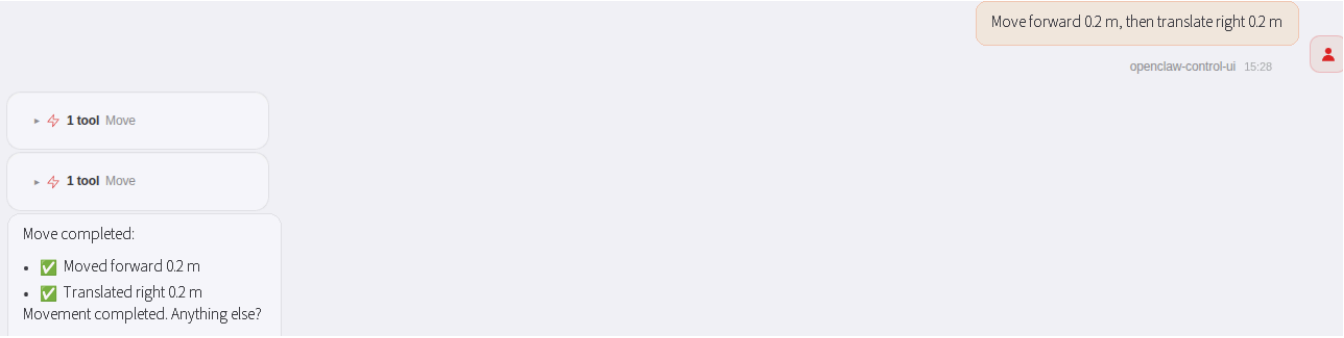
- Use any interaction method to converse with OpenClaw, here demonstrating with Web Chat



> **Tip**
>
> - You can write the conversation statements according to your own needs; following is the case demonstration

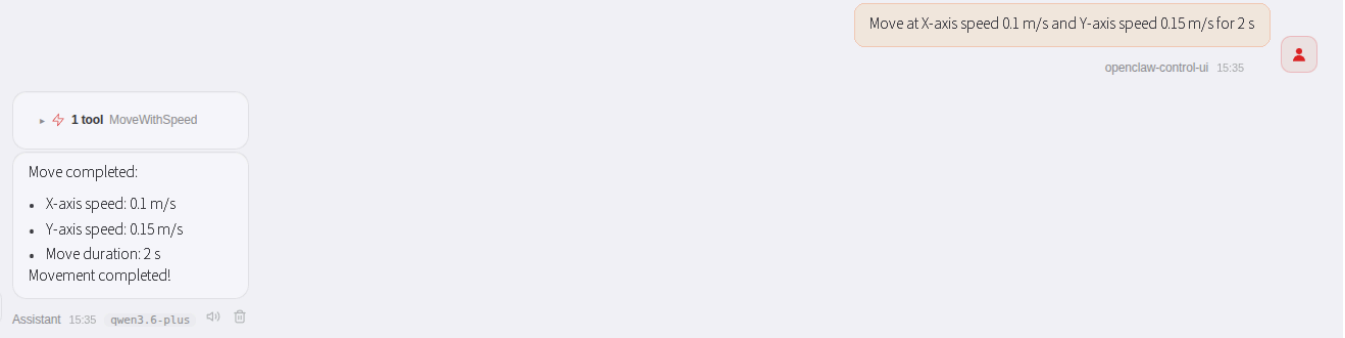
3.2 Chassis Movement Control

- Move forward 0.2 meters, then translate right 0.2 meters



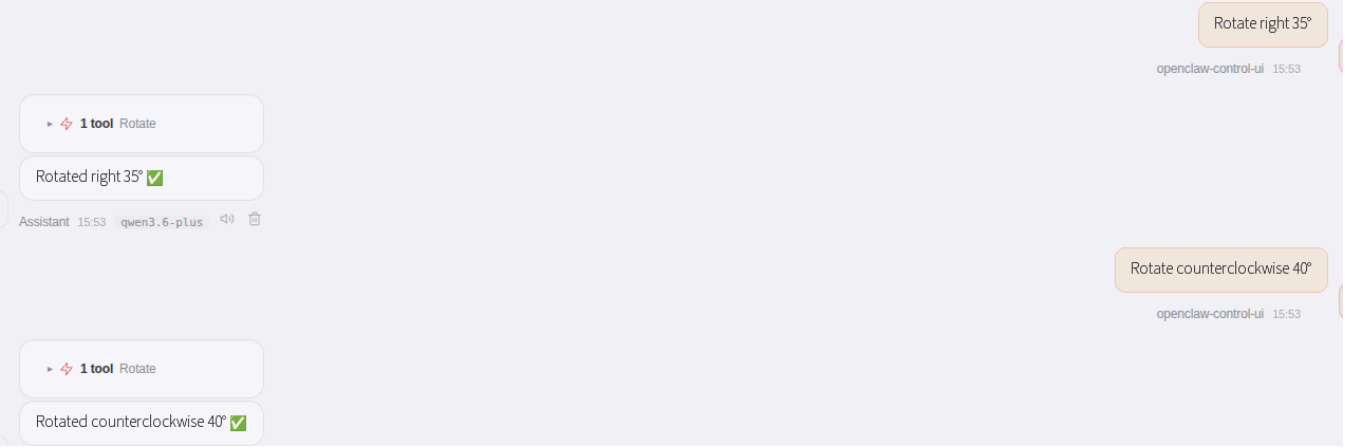
3.3 Chassis Omnidirectional Movement Control

- The MoveWithSpeed tool provides lower-level velocity control, capable of completing complex movements through vector composition of velocities, for example:
- Please move for 2 seconds at an x-axis velocity of 0.1 m/s and a y-axis velocity of 0.15 m/s



3.4 Chassis In-Place Steering

- Rotate right 35 degrees, rotate counterclockwise 40 degrees



4. Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

4.1 `set_cmdvel` — Set Chassis Motion Velocity (Underlying Implementation of MoveWithSpeed)

Code Description:

- `set_cmdvel`: Sets the chassis motion velocity and automatically stops after the specified duration; this is the underlying implementation of the `MoveWithSpeed` tool
- `Twist()`: Creates a ROS2 velocity message object, containing linear velocity `linear` (x/y/z) and angular velocity `angular` (x/y/z)
- `twist.linear.x`: Forward/backward linear velocity, unit: m/s; positive values forward, negative values backward

- `twist.linear.y`: Left/right linear velocity, unit: m/s; positive values to the left, negative values to the right (exclusive to omnidirectional chassis)
- `twist.angular.z`: Angular velocity, unit: rad/s; positive values rotate counterclockwise, negative values rotate clockwise
- `self.cmd_pub.publish(twist)`: Publishes the velocity command to the `cmd_vel` topic, driving the chassis to move at the set velocity
- `time.sleep(duration)`: Continues moving for the specified time (seconds)
- `self.cmd_pub.publish(Twist())`: Publishes an empty velocity message (all components zeroed), stopping the chassis

1. **Message construction**: Creates a `Twist()` velocity message, setting `linear.x` (forward/backward linear velocity), `linear.y` (left/right linear velocity), and `angular.z` (angular velocity)
2. **Publish and stop**: Publishes to the `cmd_vel` topic through `cmd_pub`, continues for the specified time via `time.sleep(duration)`, then publishes an empty `Twist()` to stop
3. **Omnidirectional control**: Sets the x/y linear velocities and z angular velocity simultaneously, achieving movement in any direction through velocity vector composition

```
def set_cmdvel(self, linear_x:float, linear_y:float, angular_z:float, duration:float)->bool:
    """
    Set robot movement speed and duration
    """
    twist = Twist()
    twist.linear.x = linear_x
    twist.linear.y = linear_y
    twist.angular.z = angular_z
    self.cmd_pub.publish(twist)
    time.sleep(duration)
    self.cmd_pub.publish(Twist())
    return True
```

4.2 `move_by_odom`

- The server-side program is provided by the chassis driver; program path: `~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/yahboomcar_bringup/mcnamu_driver.py`

Code Description:

- `_request_move_by_odom`: Client function that sends an odometry movement request to the mcnamu_driver MoveByOdom Action server and waits for the execution result
- `wait_for_server`: Waits for the MoveByOdom Action server to come online; the timeout is 10 seconds, and it returns False if it times out
- `MoveByOdom.Goal()`: Constructs the Action goal message, setting three parameters: `distance` (movement distance), `angle` (target angle), and `direction` (movement direction)

- `send_goal_async`: Asynchronously sends the goal to the Action server, waiting for the send result via `_wait_future` with a 10-second timeout
- `goal_handle.accepted`: Checks whether the server accepted the goal request; if rejected, logs an error and returns False
- `get_result_async`: Asynchronously obtains the Action execution result; `timeout=None` means relying on the server's own timeout limit
- `result.success`: Success flag of the execution result; `True` means the movement is complete, `False` means failure
- `result.message`: Detailed information of the execution result, containing a description of the specific reason for success or failure
- `_wait_future`: An internally encapsulated Future waiting method that supports timeout control, avoiding infinite blocking

```
    def _request_move_by_odom(self, distance: float, angle: float, direction: str) -> bool:
        '''Request the mcnamu_driver move_by_odom action service and wait for the result'''
        if not self.move_by_odom_client.wait_for_server(timeout_sec=10.0):
            self.get_logger().error("MoveByOdom action server not available")
            return False

        goal = MoveByOdom.Goal()
        goal.distance = float(distance)
        goal.angle = float(angle)
        goal.direction = direction

        send_goal_future = self.move_by_odom_client.send_goal_async(goal)
        if not self._wait_future(send_goal_future, timeout=10.0):
            self.get_logger().error("Failed to send MoveByOdom goal")
            return False

        goal_handle = send_goal_future.result()
        if not goal_handle.accepted:
            self.get_logger().error("MoveByOdom goal rejected by action server")
            return False

        result_future = goal_handle.get_result_async()
        # node is spinning in a background thread, timeout None relies on server side limits
        if not self._wait_future(result_future, timeout=None):
            self.get_logger().error("MoveByOdom action result timeout")
            return False

        result = result_future.result().result
        if result.success:
            self.get_logger().info(f"MoveByOdom succeeded: {result.message}")
            return True
        self.get_logger().warn(f"MoveByOdom failed: {result.message}")
        return False
```

> **Tip**

- For more detailed chassis control operations, refer to the navigation-transport and waste-sorting cases in 【OpenClaw Embodied Intelligence in Practice】